



CENTRO UNIVERSITARIO DE
CIENCIAS EXACTAS E INGENIERÍAS



Hilos POSIX Y Algoritmos de Planificación

Sistemas Operativos

Actividad 7 – Tarea 4

Alumno: Quijas Contreras Victor Manuel

Código: 220429771

Carrera: ICOM **Sección:** D05

Profesor: Becerra Velázquez Violeta Del Rocío

Departamento: Ciencias Computacionales

Fecha: 15/03/2026

Índice

Índice	1
Tabla de ilustraciones	1
Hilos POSIX.....	1
Historia	1
Definición	2
Características.....	2
Estados de un hilo	2
Funciones de PThreads.....	3
Atributos PThreads	4
Preguntas	4
Conclusión	5
Bibliografía.....	6

Tabla de ilustraciones

Ilustración 1. Prototipo de las funciones en lenguaje C	4
---	---

Hilos POSIX

Historia

Los hilos POSIX (Portable Operating System Interface) surgen a raíz de que cada fabricante de hardware implementaba sus propios hilos y la forma de operar con ellos, por lo que fue necesario que se estableciera un estándar de interfaz para facilitar el manejo y portabilidad de los hilos.

En 1995 se especificó la interfaz estándar IEEE 1003.1c. Las implementaciones basadas en este estándar son los hilos POSIX o Pthreads. Actualmente la mayoría de los fabricantes ofrecen en su hardware Pthreads, además de sus propias APIs (Intel, AMD). A pesar de que se creó en 1995, este estándar ha pasado por varios cambios y revisiones, la más reciente fue en el año 2008.

Hilos Posix Y Algoritmos de Planificación

Definición

Un hilo (thread) es la unidad más pequeña de ejecución que puede ser gestionada por un sistema operativo. Se le conoce como proceso ligero porque, a diferencia de los procesos tradicionales, los hilos no funcionan de forma aislada; coexisten como componentes de un proceso padre. Esto permite que múltiples hilos se ejecuten de manera concurrente, compartiendo el mismo espacio de memoria y recursos, lo que optimiza la comunicación y el rendimiento del software.

Características

Los hilos permiten la ejecución concurrente de instrucciones asociadas a diferentes funciones dentro de un mismo proceso.

Estos consumen una menor cantidad de memoria que los procesos porque comparten un mismo espacio de direcciones.

Los cambios de contextos entre hilos producen menos overhead (tiempo adicional que tarda en realizarse una operación del sistema como la llamada a una función).

Los hilos de un mismo proceso comparten los siguientes elementos:

- Código
- Variables de memoria globales
- Archivos o dispositivos que tuviera abiertos el hilo padre

Pero no comparten los siguientes elementos:

- Contador de programa
- Registros de CPU
- Pila
- Estado del hilo

Estados de un hilo

1. **Listo:** El hilo se encuentra listo para ser ejecutado, pero está esperando a que se libere un procesador.
2. **Ejecutándose:** El hilo se encuentra en ejecución.
3. **Bloqueado:** El hilo se detiene y se encuentra esperando algo, ya sea una E/S o una variable.
4. **Terminado:** El hilo termina cuando se llamó a la función *pthread_exit* o fue cancelado a mitad de ejecución.

Funciones de PThreads

- **pthread_create** crea un nuevo hilo con los atributos attr dentro del proceso que la invoca y devuelve a través de thread su identificador. Si attr vale NULL se pasan los atributos por defecto. En caso de fallo, la función devuelve un número distinto de cero que indica que error producido. El hilo ejecuta el código de la rutina referenciada por start_routine a la que se pasan los argumentos referenciados por arg.
- **pthread_self** devuelve el identificador del hilo que la invoca.
- **pthread_detach** indica al sistema que los recursos de almacenamiento ocupados por el hilo con identificador thread tienen que ser liberados cuando el hilo termine. La llamada a esta función no produce la terminación del hilo; no se recomienda llamarla más de una vez por cada hilo, de lo contrario el comportamiento es indefinido.
- **pthread_exit** termina la ejecución del hilo que la invoca y le devuelve los datos referenciados por value_ptr al hilo que esté esperando su terminación. Esta función es también llamada de forma implícita cuando termina la rutina start_routine indicada al crear el hilo
- **pthread_join** suspende la ejecución del hilo que la invoca y espera hasta que la terminación del hilo con identificador thread. El apuntador devuelto por este hilo con una llamada pthread_exit es almacenado en la dirección referenciada por value_ptr. Si un hilo intenta ejecutar pthread_join sobre sí mismo se detecta un abrazo mortal y la llamada a la función falla devolviendo el valor EDEADLK. El segundo argumento es un apuntador a apuntador porque nos interesa el valor de dicho argumento, si no fuera así se obtiene una copia de la dirección de la variable.

Hilos Posix Y Algoritmos de Planificación

- **pthread_equal** devuelve 0 si los identificadores t1 y t2 son iguales y cualquier otro valor diferente de 0 si coinciden.

```
pthread_t thread;  
int pthread_equal (pthread_t t1, pthread_t t2);  
int pthread_create (pthread_t *thread,  
    const pthread_attr_t *attr,  
    void *(*start)(void *), void *arg);  
pthread_t pthread_self (void);  
int sched_yield (void);  
int pthread_exit (void *value_ptr);  
int pthread_detach (pthread_t thread);  
int pthread_join (pthread_t thread, void **value_ptr);
```

Ilustración 1. Prototipo de las funciones en lenguaje C

Atributos PThreads

- Vinculación (detach_state)
- Tamaño de la pila (stacksize)
- Dirección de la pila (stackaddr)
- Ámbito de contienda (scope)
- Herencia de la planificación (inheritsched)
- Política de distribución (schedpolicy)
- Parámetros de planificación (schedparam)

Preguntas

1. Describa en qué consisten los algoritmos de planificación Apropiativos.

Que al llegar un proceso nuevo a listos que cumpla con la condición de prioridad de ese algoritmo de ejecución, el algoritmo va a dejar de atender a el proceso actual que esté en el procesador para comenzar a atender al nuevo que entró.

2. ¿En qué consiste el Algoritmo de Planificación por Prioridades y como se clasifica?

Atiende a los procesos tomando como prioridad alguna característica de estos, como lo podría ser su importancia, tiempo máximo estimado, u hora de llegada.

Se clasifican principalmente en dos:

Prioridad estática: La condición que prioriza no cambia y se mantiene a lo largo de toda su ejecución, puede ser apropiativa y no apropiativa.

Prioridad dinámica: La condición puede cambiar a lo largo de la ejecución del algoritmo, y puede ser apropiativo y no apropiativo.

3. ¿En qué consiste el Algoritmo de Planificación de colas múltiples?

Se utiliza para ejecutar diversas colas que desean ser atendidas, organizándolas por prioridad y un quantum que varía dependiendo su importancia. Es conocido como un RR modificado.

4. Diferencia entre el estado "Bloqueado y Suspendido" y "Listo y Suspendido".

En bloqueado y suspendido el proceso sale al disco y sigue esperando a que ocurra un evento externo. Mientras que el otro entrará a listo hasta que se libere un espacio en la RAM.

5. ¿Cuál es el tiempo de Respuesta?

Es el tiempo en el que fue atendido por primera vez un proceso.

6. Describa el algoritmo de planificación Apropiativo RR.

Atiende a los procesos por su orden de llegada, pero solo los atiende la cantidad de tiempo que marca el quantum, una vez que termina su periodo de ejecución con uno, cambia al siguiente en la cola.

7. ¿Qué es el Quantum?

Unidad de tiempo que dura ejecutándose un proceso.

Conclusión

Los hilos son una herramienta para mejorar el rendimiento de programas y procesos en general, pero a su vez, se necesita cuidar muy bien de cómo los utilizamos debido a que pueden generar una gran cantidad de conflictos como lo es la pelea de recursos de una variable, para ello existen más herramientas que se enfocan en la resolución de estos conflictos como lo pueden ser los semáforos y monitores.

Bibliografía

- ❖ Papime, P., Mondragón, A., Página, C., & Introducción, H. P. (s/f). UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO FACULTAD DE INGENIERÍA LABORATORIO INTEL PARA LA ACADEMIA. Unam.mx. Recuperado el 15 de marzo de 2026, de <http://lcomp89.fi-b.unam.mx/assets/ProgramacionConMultitareas/PracticaHilosPOSIX.pdf>
- ❖ POSIX threads in OS. (2021, junio 8). GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/posix-threads-in-os/>